

Algorytm rekurencyjny – ciąg Fibonacciego

„[] opanowanie teorii jest zaledwie niezbędnym wstępem”

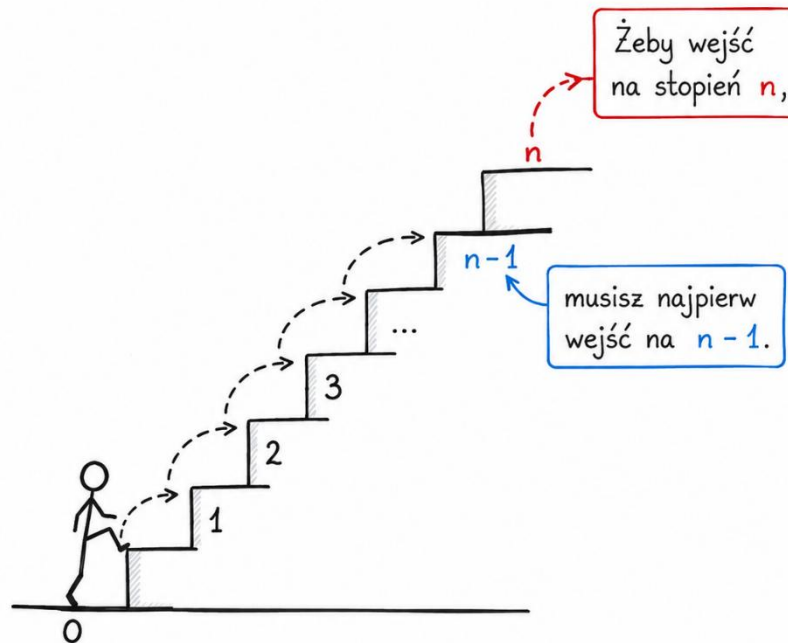
Rekurencja

Rekurencja

- **Rekurencja** to technika programowania, w której funkcja wywołuje samą siebie w celu rozwiązania problemu poprzez dzielenie go na mniejsze podproblemy – taka funkcja działa aż do osiągnięcia tzw. warunku bazowego (warunku zakończenia)

Rekurencja

Schody: żeby wejść na stopień n ,
musisz wejść na $n - 1$



Aby wejść na dowolny stopień $n \geq 1$,
trzeba najpierw wejść na stopień $n - 1$.

Ciąg Fibonacciego rekurencyjnie

Ciąg Fibonacciego rekurencyjnie

Aby obliczyć wartość 5-ego elementu ciągu należy obliczyć wartość 4-ego i 3-ego elementu oraz je zsumować

$$\text{Fibonacci}(5) \rightarrow \text{Fibonacci}(4) + \text{Fibonacci}(3)$$

Ciąg Fibonacciego rekurencyjnie

- Dla obliczenia $F(5)$ rekurencja przedstawia się następująco:

- $F(5) = F(4) + F(3)$

- $F(4) = F(3) + F(2)$

- $F(3) = F(2) + F(1)$

- $F(2) = 1$

- $F(1) = 1$

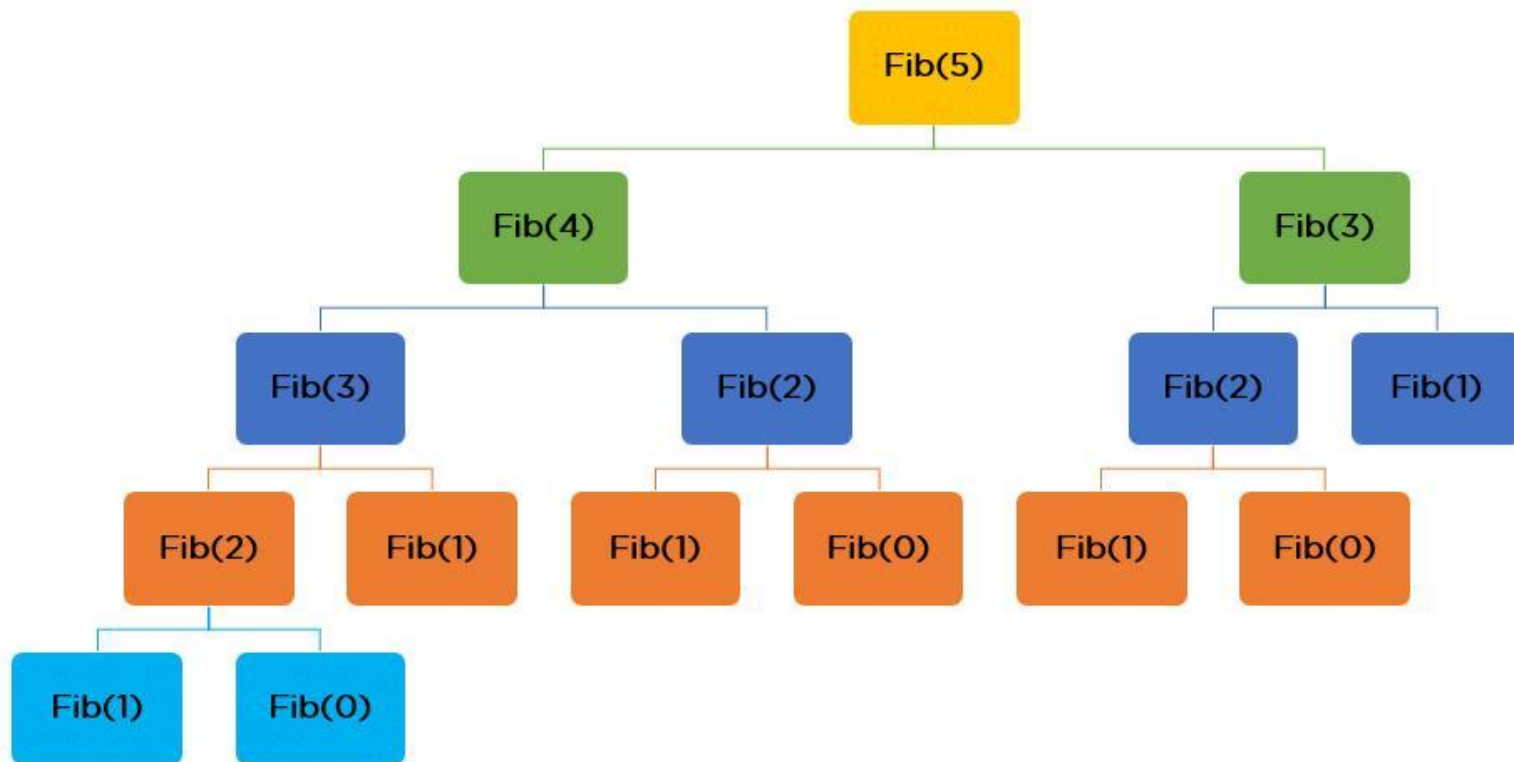
Funkcja wywołuje samą siebie,
ale zmienia się argument

Obliczenia się powtarzają

- Dzieli problem na podproblemy

Ciąg Fibonacciego rekurencyjnie

Call Stack - Computation Flow Chart



Ciąg Fibonacciego rekurencyjnie

Fibonacci(5) → Fibonacci(4) + Fibonacci(3)

Każde wywołanie (`fib(k)`) trafia na `stos` i `czeka na wynik` kolejnych wywołań.
Po otrzymaniu wyników – `oblicza swój wynik` i `wraca` (jest zdejmowane ze stosu).

Ciąg Fibonacciego rekurencyjnie

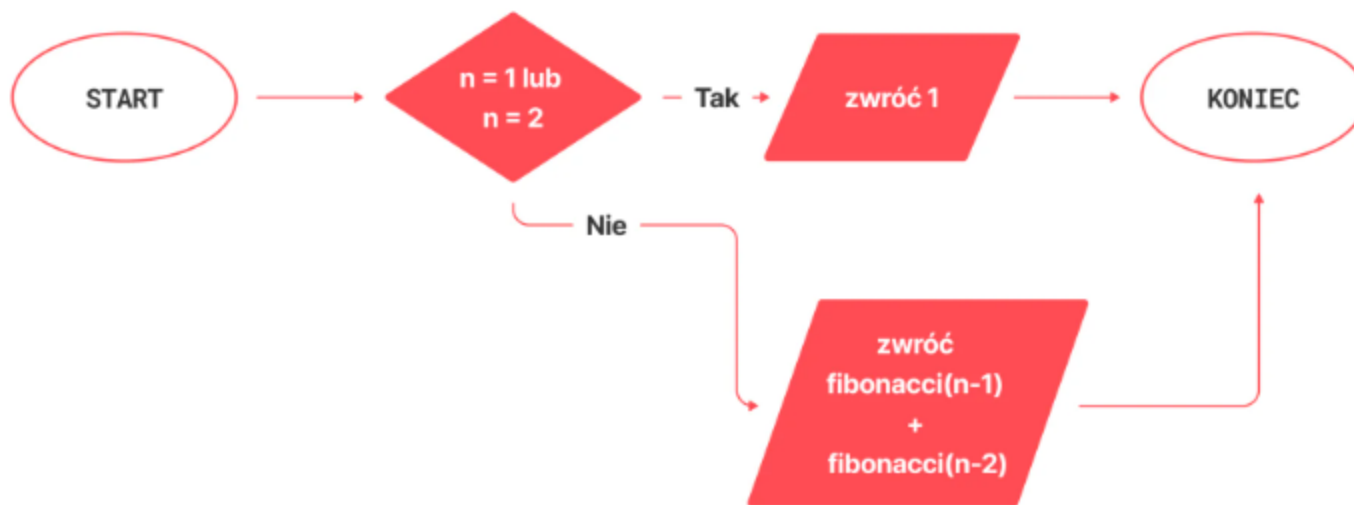
- Faza 1 – wywołania funkcji:
 - funkcja schodzi do najmniejszego elementu, tj. $F(5) \rightarrow F(4) \rightarrow F(3) \rightarrow F(2)$
- Faza 2 – powroty:
 - po osiągnięciu przypadku bazowego wyniki są zwracane i składane

Pseudokod

Pseudokod

```
funkcja Fibonaccini(n: całkowita) zwraca całkowita
    jeżeli n = 1 lub n = 2 wtedy
        zwróć 1
    w przeciwnym razie
        zwróć Fibonaccini(n - 1) + Fibonaccini(n - 2)
koniec funkcji
```

Schemat blokowy



Implementacja w C#

```
static int Fibonacci(int n) {  
    if (n == 1 || n == 2)  
        return 1;  
    else  
        return Fibonacci(n - 1) + Fibonacci(n - 2);  
}
```



```
static void Main(string[] args)
{
    int a = 5;
    int n = Fibonacci(a);
    Console.WriteLine(a + " wyraz ciągu Fibonacciego to " + n);
}
```

Cechy rekurencji

Cechy rekurencji

- Obliczenia często się powtarzają: np. $F(3)$ liczone jest wiele razy, co powoduje:
 - dużą liczbę wywołań,
 - niską wydajność (złożoność wykładnicza)

Cechy rekurencji

- Rekurencja to mechanizm, w którym:
 - funkcja wywołuje samą siebie
 - używany jest stos wywołań
 - wymagany jest przypadek bazowy
 - obliczenia są wykonywane „w dół”, a wyniki składane „w górę”

Zadanie

Zadanie

- Obliczmy sumę liczb od 1 do 5 rekurencyjnie

$$\text{Suma} = 5 + 4 + 3 + 2 + 1$$

$$\text{Suma} = 15$$



$$\text{Suma}(n) = n + \text{suma}(n-1)$$

Zadanie

Odwołania: 2

```
static int Sum(int n) {  
    if(n == 0) return 0;  
    return n + Sum(n - 1);  
}
```

www.szymkowiak.tychy.pl

KONIEC

Grzegorz Szymkowiak